

Rules grow with your controls. Tenants, documents and amounts don't add rules.

OpenBox can match a rule on any input a tool receives, but you don't have to. Most controls are one condition on the tool name, and that one rule covers every call to the tool. Reach into an input only when its value decides the outcome, such as an amount over a limit. We ran a multi-tenant agent under three policy sets to show it.

THE SETUP

Agent	Workload	Policy sets	Governed by
A LangGraph procurement agent with 3 tools: <code>read_document</code> , <code>create_purchase_order</code> , <code>send_email</code> . Every tool call carries the tenant it serves.	The same 8 calls every run, for 3 tenants (acme, globex, initech). Purchase orders of 120, 2,400, 499 and 501.	No rules. Then 1 tool-level rule. Then that rule plus 1 input-level rule.	OpenBox, through the LangGraph SDK. Every call is evaluated before it runs.

Same calls, three policy sets

CALL	INPUT	NO RULES 0 rules	TOOL LEVEL 1 rule	TOOL + INPUT LEVEL 2 rules
acme · read_document	purchase-request-1001	RAN	RAN	RAN
acme · create_purchase_order	amount 120	RAN	RAN	RAN
globex · read_document	purchase-request-2001	RAN	RAN	RAN
globex · create_purchase_order	amount 2,400	RAN	RAN	BLOCKED
initech · create_purchase_order	amount 499	RAN	RAN	RAN
initech · create_purchase_order	amount 501	RAN	RAN	BLOCKED
acme · send_email	finance@acme.example	RAN	BLOCKED	BLOCKED
globex · send_email	sales@dell.example	RAN	BLOCKED	BLOCKED
Stopped		0 of 8	2 of 8	4 of 8

The tool-level rule stopped both emails without naming a tenant or a recipient. The input-level rule split purchase orders on the amount: 499 ran, 501 was stopped. Nothing else changed.

The two rules

TOOL LEVEL · 1 CONDITION

No outbound email

activity_type is send_email
→ BLOCK

- Matches every send_email call, for any tenant, recipient or subject
- Never needs updating when a tenant or recipient is added
- Use for: "this agent may never ...", "... always needs approval"

INPUT LEVEL · 2 CONDITIONS

Purchase orders over 500

activity_type is
create_purchase_order
AND activity_input[0].amount > 500
→ BLOCK

- The same tool, split by one value the business cares about
- Any numeric field works the same way: amount, quantity, page count
- Use only when a value decides the outcome

OpenBox

Dashboard

Agents

Approvals

Identity & Access

Organization

User Guide

Documentation

aswin@openbox.ai
Administrator

Search rules...

All Decisions

All Status

RULE	CONDITIONS	DECISION	PRIORITY	TRUST	STATUS	ACTIONS
Purchase orders over 500 Input-level rule: purchase orders above 500 are stopped; smaller ones go through.	2 conditions Match all	BLOCK	60	None		...
CONDITIONS						
LEFT FIELD	LEFT OPERATION	OPERATOR	COMPARE WITH	RIGHT OPERAND		
activity_type	Value	is	Static value	create_purchase_order (string)		
activity_input[0].amount	Value	>	Static value	500 (number)		
No outbound email Tool-level rule: the agent may not send email, for any tenant or recipient.	1 condition Match all	BLOCK	50	None		...
CONDITIONS						
LEFT FIELD	LEFT OPERATION	OPERATOR	COMPARE WITH	RIGHT OPERAND		
activity_type	Value	is	Static value	send_email (string)		
Version History						
VERSION HASH	DESCRIPTION			TIMESTAMP		ACTION
52643fc	Policy Rule status changed (2 active rules)			Oct 9, 2026, 2:45 PM		Current
32c46b3	Policy Rule status changed (1 active rules)			Oct 9, 2026, 2:45 PM		Rollback
bec2658	Policy Rule status changed (0 active rules)			Oct 9, 2026, 2:43 PM		Rollback
46edd09	Policy Rule status changed (1 active rules)			Oct 9, 2026, 2:43 PM		Rollback
b1df3db	Policy Rule status changed (2 active rules)			Oct 9, 2026, 2:43 PM		Rollback
da574d1	Policy Rule status changed (1 active rules)			Oct 9, 2026, 2:41 PM		Rollback
c8f16ed	Policy Rule created (0 active rules)			Oct 9, 2026, 2:41 PM		Rollback

The agent's rules in OpenBox. One rule with 1 condition, one with 2. This is the entire policy for every tenant the agent serves.

Same intent, two shapes

Every control can be written broadly, on the tool alone, or precisely, on the tool plus one input. Here are three intents, each written both ways and run on the same 8 calls, one rule at a time. The broad shape is always one condition. The precise shape adds one more, and it is still one rule for every tenant.

INTENT 1

Stop large purchase orders

The intent names a value: the amount.

BROAD · TOOL LEVEL

```
activity_type is create_purchase_order  
→ BLOCK
```

BLOCKED acme · 120
BLOCKED globex · 2,400
BLOCKED initech · 499
BLOCKED initech · 501

4 of 4 stopped. Also stops the 120 and 499 orders, which the intent allows.

PRECISE · INPUT LEVEL

```
activity_type is create_purchase_order  
AND activity_input[0].amount > 500  
→ BLOCK
```

RAN acme · 120
BLOCKED globex · 2,400
RAN initech · 499
BLOCKED initech · 501

2 of 4 stopped. Exactly the orders over 500.

Use the precise shape. The intent is about the amount, so the rule has to read the amount.

INTENT 2

Keep email inside the tenant

The intent names a relationship between two inputs: who the email goes to, and which tenant it is for.

BROAD · TOOL LEVEL

```
activity_type is send_email  
→ BLOCK
```

BLOCKED acme → finance@acme.example
BLOCKED globex → sales@dell.example

2 of 2 stopped. Also stops acme emailing its own finance team.

PRECISE · INPUT LEVEL

```
activity_type is send_email  
AND activity_input[0].to does not contain  
activity_input[0].tenant  
→ BLOCK
```

RAN acme → finance@acme.example
BLOCKED globex → sales@dell.example

1 of 2 stopped. Only the email leaving its tenant.

Use the precise shape, and note it compares two inputs instead of listing domains. One rule covers acme, globex and every tenant added later. If the intent were "this agent never sends email", the broad shape would be the right one.

INTENT 3

Keep the agent out of Globex's documents

The intent names one tenant.

BROAD · TOOL LEVEL

```
activity_type is read_document  
→ BLOCK
```

BLOCKED acme · purchase-request-1001

BLOCKED globex · purchase-request-2001

2 of 2 stopped. Also locks acme out of its own documents.

PRECISE · INPUT LEVEL

```
activity_type is read_document  
AND activity_input[0].tenant is globex  
→ BLOCK
```

RAN acme · purchase-request-1001

BLOCKED globex · purchase-request-2001

1 of 2 stopped. Only Globex's document.

Use the **precise shape**, but only for the tenants that are exceptions. This is the one shape where a value is written into the rule, so it is the only kind that adds a rule per tenant, and only per exceptional tenant.

The screenshot shows the OpenBox interface with a sidebar on the left containing navigation links: Dashboard, Agents (highlighted), Approvals, Identity & Access, Organization, User Guide, and Documentation. The main area displays a list of rules with columns for RULE, CONDITIONS, DECISION, PRIORITY, TRUST, STATUS, and ACTIONS. Four rules are visible, each with a detailed view of its conditions.

RULE	CONDITIONS	DECISION	PRIORITY	TRUST	STATUS	ACTIONS
Purchase orders over 500 Input-level rule: purchase orders above 500 are stopped; smaller ones go through.	2 conditions Match all	BLOCK	60	None	Off	...
CONDITIONS						
LEFT FIELD	LEFT OPERATION	OPERATOR	COMPARE WITH	RIGHT OPERAND		
activity_type	Value	is	Static value	create_purchase_order (string)		
activity_input[0].amount	Value	>	Static value	500 (number)		
Email only inside the tenant Input-level shape of 'keep email inside the tenant': stops email whose recipient does not contain the tenant's name. One rule for every tenant.	2 conditions Match all	BLOCK	55	None	Off	...
CONDITIONS						
LEFT FIELD	LEFT OPERATION	OPERATOR	COMPARE WITH	RIGHT OPERAND		
activity_type	Value	is	Static value	send_email (string)		
activity_input[0].to	Value	does not contain	Another field	activity_input[0].tenant (value, string)		
No outbound email Tool-level rule: the agent may not send email, for any tenant or recipient.	1 condition Match all	BLOCK	50	None	Off	...
CONDITIONS						
LEFT FIELD	LEFT OPERATION	OPERATOR	COMPARE WITH	RIGHT OPERAND		
activity_type	Value	is	Static value	send_email (string)		
No purchase orders Tool-level shape of 'stop large purchase orders': stops every purchase order.	1 condition Match all	BLOCK	40	None	Off	...
CONDITIONS						
LEFT FIELD	LEFT OPERATION	OPERATOR	COMPARE WITH	RIGHT OPERAND		
activity_type	Value	is	Static value	create_purchase_order (string)		

The shapes as OpenBox stores them. One-condition rules match on the tool. The two-condition rules add an amount, a tenant, or a comparison with another field ("Another field": the recipient against the tenant). Each shape ran on its own, so the rules shown here are switched off.

The same six shapes in Microsoft's Agent Control Specification

Microsoft announced the Agent Control Specification (ACS) at Build 2026 as the policy layer of its open-source Agent Governance Toolkit. We wrote the same six rules as ACS policies and sent the same 8 tool calls through the reference runtime the toolkit depends on (agent-control-spec 0.4.0a4, published by Responsible AI; enforce mode). It matched OpenBox on every call.

INTENT	SHAPE	OPENBOX stopped	MICROSOFT ACS stopped	CALLS THAT DIFFER
Stop large purchase orders	Broad · tool only	4 of 4	4 of 4	none
	Precise · amount > 500	2 of 4	2 of 4	none
Keep email inside the tenant	Broad · tool only	2 of 2	2 of 2	none
	Precise · recipient vs tenant	1 of 2	1 of 2	none
Keep the agent out of Globex's documents	Broad · tool only	2 of 2	2 of 2	none
	Precise · tenant is globex	1 of 2	1 of 2	none

ONE RULE, TWO PRODUCTS

Purchase orders over 500

The same two conditions: which tool, and the value of one input.

OPENBOX RULE

```
activity_type is create_purchase_order
AND activity_input[0].amount > 500
→ BLOCK
```

MICROSOFT ACS · REGO

```
default verdict := {"decision": "allow"}

verdict := {"decision": "deny"} if {
  input.tool.id == "create_purchase_order"
  input.policy_target.value.amount > 500
}
```

Microsoft's own sample policies follow the same split. Their bank-manager example (from AgentShield) blocks freeze_account unless admin mode is on, and denies payments with amount > 50000: a rule on the tool and session state, and a rule on one input value. The Rego above reads input.tool.id because our manifest's tool catalog gives each tool an id; Microsoft's examples read input.tool.name.

How Microsoft manages policy, and where OpenBox matches it

Both products check each tool call before it runs, using rules that match on the tool and, if needed, its inputs. They differ in where the rules live and what is kept afterwards.

	OPENBOX	MICROSOFT ACS
Where policy lives	Rules on each agent, managed in the dashboard or through the rules API	A YAML manifest per agent, kept in the codebase, that binds policies to points in the agent loop
When a call is checked	Before each tool call, through the framework SDK	At up to eight points, including <code>pre_tool_call</code> , the one used here
What a rule can see	The tool name and every input (<code>activity_input</code>)	The tool name and every argument (<code>policy_target</code>), plus anything the host adds
How rules are written	Conditions chosen in a form: field, operator, value or another field	Rego or Cedar code. The toolkit also ships a simpler YAML list of name, condition, action and priority
Decisions	ALLOW, CONSTRAIN, REQUIRE_APPROVAL, BLOCK, HALT	allow, deny, transform, plus warn and escalate (deny until a person approves)
Ordering	Priority 1–100, highest number checked first; first match wins	Written into the policy, typically an <code>else</code> chain where the first match wins
When nothing matches	Allow	Whatever the policy's default says. The toolkit's YAML engine denies unless <code>default_action: allow</code> is set
Before going live	Dry-run each rule against sample inputs	<code>evaluate_only</code> mode: decide on live traffic without acting
Sharing rules across agents	No shared rule sets. Control Templates stamp the same rule onto many agents; a script or the Policy Studio can copy whole sets	<code>extends</code> : a manifest can inherit parent manifests, such as a company baseline
What is kept afterwards	Every call's inputs and verdict in a signed session record, viewable in the dashboard	Decision events that by rule leave out tool arguments. Storing and viewing them is up to you
Maturity	Running product	Draft specification, alpha runtime (0.4.0-alpha)

Why this matters

- **The concern about thousands of rules isn't specific to OpenBox.** It applies to any tool that can read inputs, and Microsoft's own model handles it the same way: one rule per control, with inputs read only when a value decides the outcome. In both products, adding tenants doesn't add rules.
- **OpenBox rules follow the model the industry is standardising on.** Each rule here translated one to one into an ACS policy and gave the same verdict on every call. Rules written for OpenBox today don't lock you into a policy model nobody else uses.
- **What the standard leaves to you, OpenBox provides.** ACS defines the decision. Where rules are stored, who can change them, how they are tested, and what record is kept are left to whoever runs it. OpenBox covers those: rules managed per agent, a dry-run before each change, and a signed record that keeps the input behind each verdict. ACS's events deliberately leave those inputs out.
- **One gap to plan for.** ACS can inherit a shared baseline from a parent manifest. OpenBox has no shared rule sets yet, so agents that share controls each get their own copy, applied from a Control Template, by script or by the [Policy Studio](#).

Adding tenants adds no rules

We reran the two-rule set with 20 more tenants. Each new tenant raised a purchase order and sent an email.

23	48	2	34
tenants	tool calls	rules, unchanged	calls stopped

All 22 emails were stopped by the tool-level rule. The 12 purchase orders over 500 were stopped and the 12 at or under 500 ran. The rule count depends on how many distinct controls the agent needs. It doesn't depend on how many tenants, documents or values flow through it. An agent serving 1,000 enterprises with these two controls still has two rules.

What OpenBox recorded

OpenBox

Dashboard
Applis
Approvals
Identity & Access
Organization
User Guide
Documentation

Terminal Window
test-ssh-input-0a11c-238ae-ru-476A-026
Copy

Terminal Window
test-ssh-input-0a11c-238ae-ru-476A-026
Copy

Event Log Timeline

Click any event to view full input/output details

All Events

Timestamp	Event Type	Activity	Verdict	Verdict Proof	Duration	Details
Oct 9, 16:43:36	WorkflowCompleted	LangGraph	ALLOW	303a6c0a8c... in-media line	174024.458ms	
Oct 9, 16:43:35	ActivityCompleted	send_email	BLOCK	39ab8e986a... in-media line	—	
Oct 9, 16:43:34	ActivityFailed	send_email	BLOCK	893a6c0a8c... in-media line	—	
Oct 9, 16:43:33	ActivityCompleted	send_email	BLOCK	76717b0a8c... in-media line	—	
Oct 9, 16:43:32	ActivityFailed	send_email	BLOCK	2986a47058... in-media line	—	
Oct 9, 16:43:31	ActivityCompleted	create_purchase_order	BLOCK	51e40c0a8c... in-media line	—	
Oct 9, 16:43:29	ActivityFailed	create_purchase_order	BLOCK	7a047b0a8c... in-media line	—	
Oct 9, 16:43:28	ActivityCompleted	create_purchase_order	ALLOW	60489a7058... in-media line	—	
Oct 9, 16:43:27	ActivityFailed	create_purchase_order	ALLOW	81e40c0a8c... in-media line	—	
Oct 9, 16:43:26	ActivityCompleted	create_purchase_order	BLOCK	53847a4058... in-media line	—	

Showing 1-12 of 18 events

Previous 1 2

OpenBox

Dashboard
Applis
Approvals
Identity & Access
Organization
User Guide
Documentation

create_purchase_order ALLOW ActivityStarted
Priority: Row
amount: 120,
email: "test@...",
sender: "Test@...",
quantity: 3
...members (1)
Copy Day view Copy overview

read_document ALLOW ActivityFailed
Priority: Row
email: "test@...",
sender: "Test@...",
quantity: 3
...members (1)
Copy Day view Copy overview

read_document ALLOW ActivityFailed
Priority: Row
email: "test@...",
sender: "Test@...",
quantity: 3
...members (1)
Copy Day view Copy overview

create_purchase_order BLOCK ActivityStarted
Priority: Row
amount: 1200,
email: "Test@...",
sender: "Test@...",
quantity: 3
...members (1)
Copy Day view Copy overview

One verdict per call. The emails and the orders over 500 are BLOCK. Every other call is ALLOW, and each event is in the session's signed record.

The input behind the verdict. The 120 order shows ALLOW and the 2,400 order BLOCK, with the amount the rule compared right there in the captured input.

Which kind of rule to write

IF THE CONTROL IS ...	WRITE	EXAMPLE
The agent may never use a tool	Tool level	<code>activity_type is send_email → BLOCK</code>
A tool always needs a person to approve	Tool level	<code>activity_type is wire_transfer → REQUIRE_APPROVAL</code>
A tool is fine up to a limit	Input level, number	<code>amount > 500, quantity >= 100</code>
A tool is fine except for certain destinations	Input level, text	<code>to contains @competitor.example, path starts_with /finance/</code>
A relationship between two inputs must hold	Input level, field to field	<code>to does not contain tenant</code>
One tenant needs a stricter rule than the rest	Input level, on the tenant field only	<code>tenant is globex AND amount > 100</code>

Start with one tool-level rule per control. Add an input condition only where the answer to "should this run?" depends on a value. Everything the tool receives is available to a rule, and none of it has to be used.

What to know

- **Rules belong to an agent.** There are no org-wide or role-wide policy rules today. Agents that should share a control set each get the same rules. That's a short script against the rules API (this demo applies its policy sets that way), and the [Policy Studio](#) (an OpenBox tool that turns an NDA into policy rules) can generate them from a contract. For a single rule, the dashboard's Control Templates stamp it onto each agent.
- **Calls that match no rule are allowed.** Write explicit BLOCK or REQUIRE_APPROVAL rules for what must not happen. Every rule needs at least one condition. To match every call, use a condition that is always true, such as `activity_type exists`.
- **Numeric comparisons need numbers.** `amount > 500` works when the tool receives a number. If a tool passes amounts as text ("100"), the policy engine sorts text above every number, so `"100" > 500` is true and every such call is blocked. Convert amounts to numbers in the tool's signature first.
- **Test a rule before it goes live.** Each rule can be dry-run against sample inputs before activation; the demo checks 499, 500 and 501. A dry-run tests one rule on its own, not its interaction with the agent's other rules.

METHOD

Run on 9 Oct 2026 against a local OpenBox stack (develop) with the OpenBox LangGraph SDK for Python. The agent makes its tool calls from a fixed script with no model involved, so every run sends identical calls and only the policy set changes. Rules were created through the OpenBox rules API, dry-run on sample inputs, activated, and confirmed loaded in the policy engine before each run. Outcomes come from the agent's own record of each call and match the verdicts OpenBox stored.

Microsoft ACS: the six rules are written as Rego policies (`acs/policy/`), each bound to `pre_tool_call` by its own manifest. The same 8 calls went through `agent-control-spec 0.4.0a4` from PyPI in enforce mode (`acs/run_acs.py`, run in a Linux x86-64 container because that is the only published wheel). The comparison table draws on the ACS specification and the Agent Governance Toolkit repository as of 9 Oct 2026.